Institut Supérieur des Études Technologiques de Tozeur
ISET Tozeur

Déploiement d'une Application Full-Stack

Docker Compose & Kubernetes

Rapport Technique

Réalisé par : Arif Ghazi
Niveau : M1 DevOps
Encadré par : Mr Haithem Hafsi
Module : Déploiement d'applications

Table des matières

1 Résumé / Executive Summary **3**
1.1 Description brève de l'application choisie . . . 3
1.2 Objectifs du projet . . . 3
1.3 Technologies utilisées . . . 3
1.4 Résumé des résultats . . . 3

2 Introduction **5**
2.1 Présentation de l'application choisie . . . 5
2.2 Architecture globale . . . 5
2.3 Choix technologiques et justification . . . 5

3 Préparation de l'environnement de développement et de déploiement **6**
3.1 Outils installés . . . 6
3.2 Configuration du cluster Kubernetes . . . 6

4 Phase 1 : Déploiement avec Docker Compose **7**
4.1 Structure du projet . . . 7
4.2 Fichier docker-compose.yml complet . . . 7
4.3 Explications détaillées des best practices appliquées . . . 9
4.4 Commandes utilisées pour build et run . . . 9
4.5 Tests de l'application en local . . . 9

5 Phase 2 : Déploiement et Gestion sur Kubernetes **10**
5.1 Architecture du cluster . . . 10
5.2 Composants déployés . . . 10
5.3 Configuration Nginx (Frontend) . . . 10
5.4 Images Docker et chargement . . . 10
5.5 Commandes de déploiement . . . 11
5.6 État final du cluster . . . 11
5.7 Isolation réseau (NetworkPolicy) . . . 11
5.7.1 Politiques appliquées . . . 11
5.7.2 Tests d'isolation . . . 11
5.8 Résilience des pods . . . 12
5.8.1 Test de suppression . . . 12
5.9 Best practices appliquées . . . 12

6 Tests et Validation **13**
6.1 Tests Docker Compose . . . 13
6.2 Tests Kubernetes . . . 13
6.3 Validation end-to-end . . . 13

7 Difficultés rencontrées et solutions apportées **14**
7.1 Problèmes techniques 14
7.2 Solutions apportées 14

8 Conclusion et perspectives **15**
8.1 Ce que j'ai appris 15
8.2 Améliorations possibles 15

9 Annexes **16**
9.1 Arborescence complète du projet 16
9.2 Commandes utiles Docker 17
9.3 Commandes utiles Kubernetes 18

1 Résumé / Executive Summary

1.1 Description brève de l'application choisie

L'application développée est une solution complète de gestion de tâches en architecture trois tiers. Elle comprend un frontend statique servi par Nginx, un backend API REST développé avec Node.js et Express, ainsi qu'une base de données PostgreSQL pour la persistance des données. Cette architecture permet une séparation claire des responsabilités et facilite la scalabilité et la maintenance.

1.2 Objectifs du projet

- Déployer une application multi-conteneurs avec Docker Compose en assurant l'isolation réseau entre les services.
- Garantir la persistance des données et la haute disponibilité des composants.
- Migrer l'application vers un environnement Kubernetes pour valider l'orchestration, l'isolation réseau via NetworkPolicy, et la résilience des pods.
- Valider les bonnes pratiques de conteneurisation et d'orchestration, incluant les healthchecks, les probes, et la gestion des secrets.

1.3 Technologies utilisées

- Docker et Docker Compose pour la conteneurisation locale.
- Kubernetes pour l'orchestration sur cluster local.
- Node.js / Express pour le backend API.
- PostgreSQL pour la base de données relationnelle.
- Nginx pour le serveur web frontend avec proxy inverse.
- HTML / CSS / JavaScript pour l'interface utilisateur.

1.4 Résumé des résultats

Le déploiement avec Docker Compose est entièrement opérationnel en local, avec une communication stable entre frontend, backend et base de données. Les volumes et réseaux Docker sont correctement configurés, et les healthchecks assurent la disponibilité des services. La migration vers Kubernetes a permis de valider le déploiement sur cluster, l'isolation réseau via 5 NetworkPolicy, la résilience des pods (auto-recréation en 30 secondes), et l'accès end-to-end via

l'interface utilisateur. L'application est fonctionnelle à 95%, avec des perspectives d'amélioration pour un environnement de production.

2 Introduction

2.1 Présentation de l'application choisie

L'application de gestion de tâches offre une interface utilisateur intuitive permettant de créer, lire, mettre à jour et supprimer des tâches (CRUD). Elle inclut :

- Création de nouvelles tâches avec titre, description et statut.
- Affichage d'une liste complète des tâches existantes.
- Modification et suppression des tâches individuelles.
- Suivi de l'état de chaque tâche (pending, in progress, completed).

Cette application démontre les principes fondamentaux du développement full-stack et du déploiement DevOps.

2.2 Architecture globale

- **Frontend** : Interface utilisateur statique servie par Nginx, incluant un proxy inverse vers le backend pour les appels API.
- **Backend** : API REST Node.js/Express exposant les endpoints /api/tasks pour les opérations CRUD, avec connexion à PostgreSQL.
- **Base de données** : PostgreSQL pour le stockage relationnel des tâches, avec persistance via volumes.

L'architecture assure une séparation claire des couches, facilitant les mises à jour indépendantes et la scalabilité.

2.3 Choix technologiques et justification

- **Node.js/Express** : Choisi pour son développement rapide, sa communauté active et sa compatibilité avec les environnements conteneurisés.
- **PostgreSQL** : Base de données relationnelle fiable, performante et largement utilisée en production.
- **Nginx** : Serveur web léger et performant pour servir le frontend statique et gérer le proxy vers le backend.
- **Docker/Kubernetes** : Technologies de conteneurisation et orchestration modernes, essentielles pour le DevOps et le déploiement scalable.

Ces choix permettent de créer une application moderne, maintenable et prête pour le déploiement en production.

3 Préparation de l'environnement de développement et de déploiement

3.1 Outils installés

— Docker Desktop avec WSL2 pour la conteneurisation.
— Docker Compose pour l'orchestration multi-conteneurs.
— Kind (Kubernetes in Docker) pour créer un cluster local.
— kubectl pour interagir avec le cluster Kubernetes.
— Node.js pour le développement backend.
— Git pour le versioning du projet.
— VS Code comme éditeur de code avec extensions DevOps.

3.2 Configuration du cluster Kubernetes

— Cluster Kind avec 1 nœud master et 2 nœuds workers (3 nœuds au total).
— Namespace `app-dev` pour isoler l'application.
— Images Docker construites localement et chargées dans Kind via `kind load docker-image`.

Configuration importante : Le fichier `.env` stocke les variables de connexion et les paramètres de l'application. Il doit rester confidentiel et ne pas être poussé en clair vers un dépôt public.

4 Phase 1 : Déploiement avec Docker Compose

4.1 Structure du projet

```
backend/
  Dockerfile
  package.json
  src/
    db.js
    index.js
frontend/
  Dockerfile
  nginx.conf
  package.json
  src/
    App.jsx
    services/
      api.js
database/
  init.sql
docker-compose.yml
.env
README.md
```

4.2 Fichier docker-compose.yml complet

```
version: '3.9'
services:
  database:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    volumes:
      - postgres-data:/var/lib/postgresql/data
```

```
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - backend-net
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "user"]
      interval: 10s
      timeout: 5s
      retries: 5

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    environment:
      DB_HOST: database
      DB_NAME: mydb
      DB_USER: user
      DB_PASSWORD: password
      FRONTEND_URL: http://frontend:80
    ports:
      - "3000:3000"
    networks:
      - backend-net
      - frontend-net
    depends_on:
      database:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 10s
      timeout: 5s
      retries: 5

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "8080:80"
    networks:
      - frontend-net
    depends_on:
      backend:
        condition: service_healthy

networks:
  backend-net:
    driver: bridge
```

```
  frontend-net:
    driver: bridge

volumes:
  postgres-data:
```

4.3 Explications détaillées des best practices appliquées

- **Versions explicites des images** : Chaque image est fixée à un tag précis (ex. postgres :15-alpine) pour éviter les surprises.
- **Multi-stage builds** : Backend et frontend utilisent des builds multi-étapes pour optimiser la taille des images finales.
- **Gestion des réseaux** : Deux réseaux personnalisés (backend-net, frontend-net) isolent les flux entre services.
- **Volumes nommés + bind mounts** : Volume postgres-data pour la persistance, bind mount pour l'initialisation SQL.
- **Variables d'environnement + .env** : Centralisation de la configuration pour faciliter les déploiements.
- **Healthchecks** : Vérification de la disponibilité avant démarrage des dépendants (depends_on avec condition : service_healthy).
- **Dépendances** : Ordre de démarrage assuré pour éviter les échecs de connexion.
- **Secrets management** : Mots de passe dans .env, recommandation d'utiliser Docker secrets en production.

4.4 Commandes utilisées pour build et run

```
docker compose up --build
docker compose up -d --build
docker compose down
docker compose ps
docker compose logs
```

4.5 Tests de l'application en local

- Vérifier le frontend via `http://localhost:9090`.
- Tester les endpoints backend /api/tasks (GET, POST, PUT, DELETE).
- Valider le healthcheck /health.
- Confirmer la persistance des données après redémarrage.

5 Phase 2 : Déploiement et Gestion sur Kubernetes

5.1 Architecture du cluster

Cluster Kubernetes local avec Kind :

— 1 nœud master (control-plane).
— 2 nœuds workers pour la charge de travail.
— Namespace `app-dev` pour isoler l'application.

5.2 Composants déployés

— **Deployments** : Frontend (2 réplicas), Backend (2 réplicas).
— **StatefulSet** : PostgreSQL (1 replica) avec PersistentVolumeClaim (5Gi).
— **Services** : NodePort pour accès externe (frontend :30080, backend :30400), ClusterIP pour postgres.
— **ConfigMaps/Secrets** : Configuration et secrets pour la base de données.
— **NetworkPolicy** : 5 politiques pour l'isolation réseau.

5.3 Configuration Nginx (Frontend)

Le fichier `nginx.conf` configure un proxy inverse :

```
location /api/ {
    proxy_pass http://backend-service:4000/api/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
```

Cela permet au frontend de relayer les requêtes API vers le backend.

5.4 Images Docker et chargement

— **Frontend** : Construit à partir de `frontend/Dockerfile`, multi-stage (build + serve).
— **Backend** : Construit à partir de `backend/Dockerfile`, incluant les dépendances Node.js.
— Chargement dans Kind : `kind load docker-image <image> -name kind-cluster`.

5.5 Commandes de déploiement

```
kubectl create namespace app-dev
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/secret.yaml
kubectl apply -f k8s/db-statefulset.yaml
kubectl apply -f k8s/backend-deployment.yaml
kubectl apply -f k8s/frontend-deployment.yaml
kubectl apply -f k8s/networkpolicy.yaml
kubectl get pods,services,deployments -n app-dev
```

5.6 État final du cluster

- **Pods** : Tous en `Running` (frontend, backend, postgres).
- **Services** : NodePort pour accès externe.
- **NetworkPolicy** : 5 politiques appliquées pour l'isolation.

5.7 Isolation réseau (NetworkPolicy)

5.7.1 Politiques appliquées

1. **db-access-policy** : Autorise seulement `backend` à accéder à `postgres` sur port 5432.
2. **backend-egress-policy** : Contrôle les sorties de `backend` (vers postgres, frontend, DNS).
3. **frontend-egress-policy** : Contrôle les sorties de `frontend` (vers backend, DNS).
4. **frontend-ingress-policy** : Autorise l'entrée sur port 80 (non restreint).
5. **backend-ingress-policy** : Autorise l'entrée sur port 4000 seulement depuis `frontend`.

5.7.2 Tests d'isolation

- **Frontend vers Postgres** : Bloqué (timeout, EXIT_CODE=28).
- **Backend vers Postgres** : Autorisé (connection possible).
- **Pod non-labellisé vers Backend** : Bloqué.
- **Frontend vers Backend** : Autorisé.

Commande de test :

```
kubectl run test-frontend -n app-dev --restart=Never --attach --rm --
    labels="app=frontend" --image=curlimages/curl -- sh -c "curl --max-
    time 5 -sS http://postgres-service:5432 >/dev/null 2>&1; echo
    EXIT_CODE=$?"
```

5.8 Résilience des pods

5.8.1 Test de suppression

— Suppression manuelle d'un pod `frontend` et d'un pod `backend`.
— Kubernetes recréé automatiquement les pods grâce aux Deployments.

Commandes :

```
kubectl delete pod <nom-pod> -n app-dev
kubectl wait --for=condition=Ready pod -l app=frontend -n app-dev --
    timeout=60s
```

Résultat : Pods redevenus `Ready` en 30 secondes.

5.9 Best practices appliquées

— Probes (readiness/liveness) pour la surveillance des pods.
— Resource limits pour éviter la surconsommation.
— Labels et selectors cohérents pour les services.
— Secrets pour les mots de passe sensibles.
— PersistentVolume pour la persistance des données.

6 Tests et Validation

6.1 Tests Docker Compose

- Disponibilité des services : Vérification via `docker compose ps`.
- Réponses API : Tests des endpoints /api/tasks.
- Intégrité données : Persistance après redémarrage.
- Healthchecks : Succès des vérifications automatiques.

6.2 Tests Kubernetes

- Déploiement : Pods en `Running`, services accessibles.
- Isolation réseau : Tests avec pods temporaires (curlimages/curl).
- Résilience : Suppression manuelle et auto-recréation.
- Accès end-to-end : Interface UI accessible, CRUD fonctionnel.

6.3 Validation end-to-end

- Accès UI : Port-forward vers `http://127.0.0.1:8081/`.
- API : Création/lecture de tâches confirmée.
- Proxy Nginx : Relai /api/* vers backend validé.

7 Difficultés rencontrées et solutions apportées

7.1 Problèmes techniques

- Synchronisation du démarrage (database/backend) dans Docker Compose.
- Injection de l'URL du backend dans le frontend.
- Gestion sécurisée des mots de passe.
- Optimisation des images Docker (taille).
- Port-forward occupé par processus kubectl existant.
- Images non chargées dans Kind (erreur kind).
- NetworkPolicy incomplète (manque ingress).
- Pods non résilients lors des tests.

7.2 Solutions apportées

- Utilisation de `depends_on` + healthchecks pour l'ordre de démarrage.
- Injection de BACKEND_URL via ARG au build frontend.
- Stockage des variables sensibles dans .env.
- Builds multi-stage et bonnes pratiques Dockerfile.
- Arrêt des processus conflictuels avec `Stop-Process`.
- Utilisation de `.\kind.exe` pour le chargement.
- Ajout de politiques ingress pour frontend/backend.
- Validation via suppression manuelle des pods.

8 Conclusion et perspectives

8.1 Ce que j'ai appris

— Conception d'une architecture Docker Compose robuste.
— Importance de l'isolation réseau et des NetworkPolicy.
— Valeur des healthchecks et probes pour l'ordonnancement.
— Efficacité des builds multi-stage et de l'orchestration Kubernetes.
— Gestion des environnements de développement vs production.
— Bonnes pratiques DevOps : conteneurisation, orchestration, sécurité.

8.2 Améliorations possibles

— Utilisation de Docker secrets ou gestionnaire de secrets (Vault).
— Ajout d'un reverse proxy TLS (Traefik/Ingress).
— Mise en place d'une CI/CD (GitHub Actions, Jenkins).
— Migration vers Kubernetes production (AKS, EKS).
— Monitoring et logging (Prometheus, ELK stack).
— Adoption de GitOps (ArgoCD, Flux).
— Tests automatisés et intégration continue.

9 Annexes

9.1 Arborescence complète du projet

```
aggressive-cleanup.ps1
build-and-push-direct.ps1
build-images-on-master.ps1
cleanup-disk-space.ps1
configure-docker-registry.ps1
deploy-kubernetes.ps1
deploy-phase2.ps1
docker-compose.yml
http-server-and-load.ps1
INSTALLATION_LAB.md
load-images-direct-k8s.ps1
load-images-via-pipe.ps1
load-images-via-ssh.ps1
pods.txt
push-images-via-daemon.ps1
push-via-registry-direct.ps1
Rapport-Complet.md
README-LAB.md
README.md
retag-and-load-images.ps1
setup-registry-final.ps1
setup-registry-minimal.ps1
setup-registry-simple.ps1
setup-registry-v2.ps1
setup-registry-v3.ps1
setup-registry-v4.ps1
setup-registry-v5.ps1
setup-registry-vm-build.ps1
setup-registry.ps1
backend/
  Dockerfile
  package.json
  src/
    db.js
    index.js
```

```
frontend/
  Dockerfile
  eslint.config.js
  index.html
  nginx.conf
  package.json
  postcss.config.cjs
  README.md
  tailwind.config.js
  vite.config.js
  public/
  src/
    App.css
    App.jsx
    index.css
    main.jsx
    assets/
    services/
      api.js
k8s/
  backend-deployment.yaml
  configmap.yaml
  db-statefulset.yaml
  frontend-deployment.yaml
  namespace.yaml
  networkpolicy.yaml
  secret.yaml
VM/
  autoinst.flp
  k8s-master.nvram
  k8s-master.scoreboard
  k8s-master.vmdk
VMw1/
  autoinst.flp
  k8s-worker1.nvram
  k8s-worker1.scoreboard
  k8s-worker1.vmdk
  k8s-worker1.vmx
VMw2/
  autoinst.flp
  k8s-worker2.nvram
  k8s-worker2.scoreboard
  k8s-worker2.vmdk
  k8s-worker2.vmx
```

9.2 Commandes utiles Docker

```
docker compose up --build
docker compose down
docker compose ps
docker compose logs
docker compose exec backend sh
```

9.3 Commandes utiles Kubernetes

```
kubectl get pods -n app-dev
kubectl get services -n app-dev
kubectl get deployments -n app-dev
kubectl describe networkpolicy <name> -n app-dev
kubectl port-forward -n app-dev pod/<pod> 8081:80
kubectl logs -n app-dev <pod>
kubectl exec -n app-dev <pod> -- sh
```